TECHNICAL DOCUMENTATION

RAG-Based Customer Support Assistant

with LangGraph & Human-in-the-Loop (HITL)

Internship Project | 2026

1. Introduction

1.1 What is RAG?

Retrieval-Augmented Generation (RAG) is an AI architecture that combines information retrieval with large language models (LLMs). Instead of relying on the LLM's training data alone — which may be outdated or too general — RAG first retrieves relevant text from a custom knowledge base and then uses the LLM to generate an accurate, grounded answer based on that retrieved content.

This approach solves two critical problems with LLMs: hallucination (making up facts) and knowledge cutoff (not knowing recent or private data).

1.2 Why RAG is Needed

- LLMs have a knowledge cutoff date and cannot access company-specific documents
- Without grounding, LLMs hallucinate and produce confident but incorrect answers
- RAG allows organizations to build AI systems on their own private data
- It is cost-effective — no need to fine-tune or retrain the model

1.3 Use Case Overview

This project implements a Customer Support Assistant for a fictional e-commerce company (ShopEasy). The system reads a PDF FAQ document and answers customer queries such as return policies, refund timelines, and contact information. Complex queries (legal complaints, fraud reports) are escalated to a human agent via the HITL module.

2. System Architecture Explanation

2.1 Ingestion Pipeline

The ingestion pipeline runs once to prepare the knowledge base. The PDF is loaded using PyPDFLoader which extracts text page by page. The extracted text is then split into small overlapping chunks using RecursiveCharacterTextSplitter. Each chunk is passed through the HuggingFace all-MiniLM-L6-v2 model to produce a 384-dimensional embedding vector. These vectors are stored in ChromaDB — an open-source vector database — enabling fast similarity search.

2.2 Query Pipeline

When a user submits a query, the LangGraph workflow is invoked. The retrieve node converts the query to an embedding and searches ChromaDB for the top-2 semantically similar chunks. The route node inspects the retrieved context and the query to decide whether to generate an automated answer or escalate to a human. If automated, the answer node sends the context and question to the Groq-hosted LLaMA 3.1 LLM which formulates a natural language response. If escalated, the hitl node collects a human agent's response.

2.3 Component Interactions

LangGraph acts as the orchestrator. Each node reads from and writes to a shared State object — a typed dictionary that carries the question, retrieved context, final answer, and escalation

flag. Nodes are pure functions that receive state and return updated state. Edges connect nodes, and conditional edges implement branching logic based on state values.

3. Design Decisions

3.1 Chunk Size Choice

A chunk size of 100 characters was chosen because the knowledge base consists of short Q&A pairs. This keeps each Q&A together as a single chunk, making retrieval precise. Larger chunk sizes (e.g., 500) would merge multiple Q&As into one chunk, making the retriever return noisy, unfocused context.

3.2 Embedding Strategy

The all-MiniLM-L6-v2 model was selected because it is lightweight (80MB), runs locally without an API key, is optimized for semantic similarity tasks, and produces high-quality 384-dimensional embeddings. It was fine-tuned specifically for sentence similarity — exactly what RAG retrieval requires.

3.3 Retrieval Approach

Cosine similarity search is used with k=2 (top 2 chunks returned). For a small FAQ document this is sufficient. In production with hundreds of pages, k could be increased to 4-5. Returning fewer chunks keeps the LLM prompt concise and reduces cost and latency.

3.4 Prompt Design Logic

The prompt explicitly instructs the LLM to answer ONLY using the provided context and to say it does not have the information if the context is insufficient. This prevents hallucination — the most common failure mode in RAG systems. The prompt is intentionally minimal to reduce token usage.

4. Workflow Explanation

4.1 LangGraph Usage

LangGraph's StateGraph is used to define a directed graph where each node is a Python function. The graph is compiled into an app object using app = graph.compile(). The workflow is invoked by calling app.invoke(initial_state). LangGraph handles the execution order, state passing between nodes, and conditional branching automatically.

4.2 Node Responsibilities

retrieve_node: Receives the user question from state. Calls retriever.invoke(question) to get top-k chunks. Joins chunk texts into a single context string. Returns updated state with context field populated.

route_node: Reads context and question from state. Applies escalation rules. Returns state with needs_human boolean flag.

answer_node: Constructs a prompt combining context and question. Calls LLM with the prompt. Extracts response.content and returns it as the answer.

hitl_node: Prints the escalation message and question. Collects human agent input. Returns human's response as the answer.

4.3 State Transitions

Initial state: {question: user_input, context: '', answer: '', needs_human: False}

After retrieve_node: context field is populated with relevant chunks.

After route_node: needs_human field is set to True or False.

After answer_node or hitl_node: answer field is populated with the final response.

5. Conditional Routing Logic

5.1 Intent Detection

The routing logic uses two signals: context quality and query complexity. Context quality is measured by the length of retrieved text — very short context means the document does not contain relevant information. Query complexity is measured by checking for keywords that indicate the query requires human judgment rather than document lookup.

5.2 Routing Decisions

If either the context is too short (fewer than 20 characters) or the query contains complex keywords (legal, lawsuit, fraud, complaint, manager, refund denied), the graph routes to the HITL node. Otherwise it routes to the answer node. This logic is implemented in the should_escalate function which returns either 'hitl' or 'answer' as a string.

6. HITL Implementation

6.1 Role of Human Intervention

Human-in-the-Loop is a design pattern where an AI system recognizes the limits of its own capabilities and delegates specific decisions to a human. In this system, HITL handles two cases: queries that the knowledge base cannot answer (missing context), and sensitive queries requiring human judgment (legal or fraud issues).

6.2 Benefits

- Prevents the LLM from fabricating answers when context is missing
- Handles edge cases and sensitive situations that AI should not handle alone
- Builds user trust by providing a clear escalation path
- Allows continuous improvement — human responses can be added to the knowledge base

6.3 Limitations

- Introduces latency when human response is required
- Requires 24/7 human availability for real-time support
- Keyword-based escalation is simplistic — may miss some complex queries

- In this implementation, escalation is simulated via CLI (not a real ticket system)

7. Challenges & Trade-offs

Trade-off	Option A	Option B
Retrieval accuracy vs speed	High k (more chunks, better coverage)	Low k (faster, less noise) — chosen
Chunk size vs context quality	Large chunks (more context, less precise)	Small chunks (precise, may split sentences) — chosen
Local vs cloud embeddings	Cloud API (accurate, requires internet)	Local model (offline, slightly less accurate) — chosen
Cost vs performance	GPT-4 (high quality, costly)	LLaMA 3.1 via Groq (free, good quality) — chosen

8. Testing Strategy

8.1 Testing Approach

The system is tested using known queries against a manually created FAQ document. Since the document content is known, expected answers are predetermined and can be compared directly against system output. This is called black-box functional testing.

8.2 Sample Queries & Expected Results

Query	Expected	Route
How do I return a product?	30 days with receipt	Answer Node
How long does a refund take?	5-7 business days	Answer Node
How do I contact support?	Email / phone details	Answer Node
I want to file a legal complaint	Escalated to human	HITL Node
What is the weather today?	Escalated (no context)	HITL Node

9. Future Enhancements

9.1 Multi-Document Support

The current system loads a single PDF. Future versions could support multiple PDFs by assigning source metadata to each document and implementing source-level filtering during retrieval using ChromaDB's where clause.

9.2 Feedback Loop

Human agent responses to escalated queries can be added back to the knowledge base as new Q&A entries. Over time this reduces escalation frequency and improves automated answer coverage.

9.3 Memory Integration

Currently each query is stateless — the system has no memory of previous interactions. Integrating LangGraph's checkpointing or a conversation buffer memory would enable multi-turn conversations where the user can follow up on previous questions.

9.4 Deployment

The CLI interface can be replaced with a web API using FastAPI. The ChromaDB in-memory store can be replaced with a persistent cloud vector database (Pinecone, Weaviate). The Groq API can be swapped for any OpenAI-compatible endpoint. Docker containerization would enable cloud deployment on AWS, GCP, or Azure.

9.5 Confidence Scoring

A more sophisticated routing mechanism could use the LLM's own confidence assessment. The system could ask the LLM to rate its confidence (1-10) before answering, and automatically escalate if the score falls below a threshold.